

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediene

USTHB — Bab Ezzouar, Alger

Faculté d'Électronique et d'Informatique

Département d'Informatique

RAPPORT DE TRAVAUX PRATIQUES

Système de Communication Sécurisé

Diffie-Hellman + AES

Module : Cryptographie et Sécurité Informatique

Niveau : Master 1 — Informatique

Année universitaire : 2025 / 2026

Réalisé par :

YEDDOU Abdelkader Raouf

Mai 2026

1. Introduction

Le projet illustre le schéma classique : deux parties (Alice et Bob) conviennent d'un secret partagé via Diffie-Hellman sans transmettre ce secret en clair sur le canal, puis dérivent une clé AES pour protéger un message texte.

L'implémentation principale est une application web (web/) : génération des clés DH, dérivation HKDF, chiffrement et déchiffrement AES, avec une interface et un diagramme de séquence animé (public vs secret).

Objectifs du TP :

- Échange de clés Diffie-Hellman (DH)
- Chiffrement symétrique AES à partir du secret partagé

2. Correspondance avec le travail demandé

Exigence	Réalisation
1. Implémenter un chiffrement AES	AES conforme FIPS-197 (SubBytes, ShiftRows, MixColumns, expansion de clé pour 128/192/256 bits) dans web/aes.js. Mode CTR (IV aléatoire 16 octets). Aucune primitive AES native du navigateur.
2. Générer une clé DH	Groupe RFC 3526 MODP14 (premier p 2048 bits, g=2), exposant privé aléatoire, clé publique $g^x \bmod p$, secret partagé $B^a \bmod p / A^b \bmod p$ dans web/dh-modp.js (BigInt).
3. Transformer le secret DH en clé AES (128/192/256 bits)	Secret DH sérialisé en 256 octets. HKDF-SHA256 (via Web Crypto) avec label fixe produit 16, 24 ou 32 octets selon le choix dans l'interface (web/app.js).
4. Chiffrer et déchiffrer un message texte	Message en UTF-8 côté Alice, chiffré en AES-CTR, résultat en base64. Bob déchiffre et affiche le texte (web/app.js, champs de l'index.html).

3. Partie 1 — Implémentation de Diffie-Hellman

3.1 Paramètres du groupe

Premier p et générateur g = 2 : groupe MODP 2048 bits (groupe 14) défini dans la RFC 3526, constante codée en hexadécimal dans dh-modp.js.

3.2 Génération de clés

Pour chaque partie :

- Tirage d'un exposant privé aléatoire dans un intervalle adapté à p.
- Calcul de la clé publique : $Y = g^x \bmod p$ (modPow par exponentiation binaire).

3.3 Secret partagé

Après échange des clés publiques (simulé dans la page) :

- Alice calcule $K = Y_B^{x_A} \bmod p$
- Bob calcule $K = Y_A^{x_B} \bmod p$

Les deux valeurs sont identiques ; c'est le secret DH brut (représenté en octets de longueur fixe pour la suite du pipeline).

3.4 Fichiers concernés

- web/dh-modp.js — cœur mathématique DH.
- web/app.js — orchestration Alice/Bob, affichage des clés publiques en base64.

4. Partie 2 — Intégration AES

4.1 Chaîne « secret DH → clé AES »

Le secret brut n'est pas utilisé directement comme clé AES : on applique HKDF-SHA256 (bonne pratique) pour obtenir une clé de longueur exacte 128, 192 ou 256 bits, selon le menu de la page.

4.2 AES implémenté dans le projet

- Algorithme de bloc AES complet (chiffrement et déchiffrement de bloc).
- Mode d'opération pour les messages : CTR, adapté aux messages de longueur quelconque (xor avec un flux de blocs chiffrés).

Format du message chiffré : IV (16 octets) || ciphertext (encodé en base64 dans l'interface).

4.3 Fichiers concernés

- web/aes.js — AES + CTR.
- web/app.js — HKDF puis appels aesCtrEncrypt / aesCtrDecrypt.

5. Interface et pédagogie

- web/index.html + web/styles.css — Mise en page, choix de la taille de clé AES, zones Alice/Bob.
- web/sequence-visual.js — Diagramme Alice — canal public — Bob animé : ce qui transite en public (clés publiques, ciphertext) vs ce qui reste secret (exposants privés, clé AES, texte clair).

- Au survol, des infobulles rappellent le contenu affiché ou une explication pour les secrets non affichés.

Lancement recommandé : serveur HTTP local sur le dossier web/ (ex. `npm start` ou `npx serve web`), car Web Crypto exige un contexte sécurisé (HTTPS ou localhost).

6. Limites assumées (honnêteté scientifique)

- AES-CTR assure surtout la confidentialité. Il n'y a pas de tag d'authentification intégré (contrairement à AES-GCM). En production, on combinerait souvent CTR avec un MAC (ex. HMAC) ou un mode AEAD.
- HKDF est réalisé avec Web Crypto : l'implémentation maison porte surtout sur DH et AES ; la dérivation reste standardisée et auditée côté navigateur.
- Le scénario ne couvre pas l'authentification des parties ni la résistance à un homme du milieu (il faudrait des signatures ou TLS en plus du DH nu).

7. Conclusion

Le livrable couvre l'objectif du TP : échange de clés DH explicite (MODP14) + AES implémenté (FIPS-197) en mode CTR + dérivation HKDF vers des clés 128/192/256 bits + chiffrement et déchiffrement de texte dans une interface web pédagogique.

8. Annexe — Structure des fichiers (web/)

Fichier	Rôle
index.html	Structure de la page, textes d'aide.
styles.css	Apparence (dont diagramme de séquence).
app.js	Protocole complet côté client (DH, HKDF, AES-CTR, UI).
dh-modp.js	Diffie-Hellman (MODP14, BigInt).
aes.js	AES bloc + mode CTR.
sequence-visual.js	Animation et infobulles du diagramme.
package.json	Script de service statique éventuel (type: "module").